

CANDY

DIALOGUE ENGINE

Choice UI Customization - Advanced

1.1.0

Before proceeding further, make sure you read the [Player Choices](#) and the [Choice UI Customization - Basic](#) guides and that you have a comfortable understanding of the `§Choice_List` command and the various pieces of data it `§Choice_List` lines contain. Understanding the `§Choice_Status` and `§Timer_Status` commands is also recommended.

The Basic Choice UI Customization guide explained how to customize Choice UI scenes for their classic use. This guide offers suggestions on how further customize Choice UI scenes for more advanced behavior.

Show Older Menus

When a choice or a timer triggers a new `§Choice_List` command, a new Choice Menu is created, and the first one is not deleted. By default, the first menu will be made invisible, provided its `never_hide` variable is set to 'false'.

The `never_hide` variable is exported, so each scene that has the script attached can have its own value for it. However, instances of the same scene will have the same value, which can be a problem when different lists use the same Menu scene but should have different `never_hide` values.

One option is to use a Setup transition in a `§Choice_List` command to change the `never_hide` value for that command's value.

Another option can also be to add logic inside the `_ready()` function of [Choice_Menu.gd](#), so that it changes the `never_hide` value based on the `§Choice_List`'s general 'Custom' data.

Timer Display

The `§Choice_List` command creates and starts timers automatically, but doesn't contain any code to display those timers visually.

If you'd like to do so, you will need to add nodes to the Menu scene to display timer values either in text form, or perhaps as a progress bar or some other visual method.

You will also need to write code inside [Choice_Menu.gd](#) to make those timer display nodes properly display the countdown value, and update it in real time.

This is all dependent upon your intentions: you may want to display all timers, only specific timers, or a limited number of them. You may also want timer display to be conditional, based on variables at runtime. Candy DE can't anticipate any of that, which is why it's left up to you, but it shouldn't be difficult to implement.

The only question that remains is how or when to get your code for displaying timers to run. The best option is probably to use the `x_initialization_finished()` hook function, since it's called immediately after timers have been setup and started: just add a call to your timer display code inside of it.

Choice Button Alternatives

Choices are buttons by default because that's what most developers or players probably expect. But in actuality, these buttons can be replaced by anything that can call the `_choice_selected()` function (in `Choice_Button.gd`).

`_choice_selected()` is the function that tells the dialogue engine that a choice was made. So anything that calls this function is effectively a choice.

For some examples, you can call the function through an automated script, when the mouse enters an area, when nodes collide or overlap, when the player writes and submits text, when keys are pressed, when a timer reaches 0, and so on.

Non-Choice Events

As a reminder, when a `$Choice_List` command runs, it instantiates a Choice Menu, then waits for a choice to be made or for a timer to expire. Both of these events trigger the `choice_list_event()` signal, which is declared inside `Candy_Engine.gd` (not in the choice Menu, Category or Button scene).

However, nothing is stopping you from adding code to the Menu, Category or Button scripts to emit the signal at other timers. For example, you could write code that emits the signal when a timer's countdown reaches 10 seconds, to trigger a prompt urging the player to hurry.

The `$Choice_List` command (in `Candy_Engine.gd`) uses match statements to determine what to do when the signal is emitted, based on the signal's arguments. So you'll need to add cases to these statements, with the appropriate code for your purposes.

Of course, if the behavior you want to trigger doesn't need to rely on the dialogue engine, you could also make your code trigger that behavior directly instead of emitting the `choice_list_event()` signal. Keep in mind that whatever option you choose, you might want to pause all active timers while the custom behavior resolves (the `$Choice_List` command includes code for that, which you can copy or use as a template), and perhaps prevent the player from selecting choices as well.

Multiple Category Display

By default, only one Category in any choice Menu is ever displayed at a time. However, it should be possible to display multiple categories at once inside the same menu, if you design the menu's layout that way, and if you add custom code to manage this.

We're not going to provide an example on how to achieve this: Candy DE is designed to make this possible, but it's not intended as a built-in feature. But if you're creative, you can probably accomplish this with the help of the `x_initialization_finished()` function and the "Custom" data of either the Menu, specific Categories, or even Choices. Modifying the `$Choice_List` command code in shouldn't `Candy_Engine.gd` even be necessary unless you have a very special use-case.

As for the usefulness of this: you might want to create a Choice Menu that looks like a browser: one category is always visible, and its choices open other categories in a separate panel.

Multiple Menu Display

Likewise, you might want to display two or more Choice Menus at once on screen. You'll need to be very careful with this, because the dialogue engine allows nesting, but not simultaneous forking.

In simple terms, each `$Bridge` transition adds a nesting level. The first nesting level starts at 0, and nesting depth increases by 1 for each consecutive `$Bridge` command. When returning from a `$Bridge`, the nesting depth reduces by 1.

You can create infinite consecutive nesting levels (e.g. $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow \text{etc.}$), but you can't create two parallel nesting levels at the same depth at the same time, such as level 3 splitting into two level 4s simultaneously.

On top of that, once a `$Choice_List` command runs a `$Bridge` transition, it waits for the dialogue to return and will ignore the `choice_list_event()` signal.

We haven't tested this because it's not an intended feature, but if you tried it, it's likely that the choice buttons in the 'older' Menu would simply have no effect when clicked (the good news is, nothing would break or crash, although we're not 100% certain). So at best, the 'older' menu would be visible on screen but effectively inactive, while the newest menu would work properly.

However, you could add custom code to the older menu, which could trigger effects and behaviors without relying on the `choice_list_event()` signal. That could serve a useful purpose depending on your objectives, but you must be careful that it doesn't conflict with the new menu.

Keep in mind that while the dialogue engine has built-in logic to kill older nesting levels and start fresh at the current level (this is used by the `$Jump` command), there is no built-in logic to kill newer levels from an older level.

As an example, a `$Jump` command at level 5 will kill levels 0-5, then start fresh at level 6 (which actually becomes the new level 0). But there is no way for a command at level 3 to kill levels 4+. Implementing such logic might be possible, but it would require modifying the core engine code and could be quite complex, so we don't advise trying unless you're highly experienced.